

Atividade Objetiva 3

Entrega 2 jun em 23:59 **Pontos** 1 **Perguntas** 5 **Disponível** 10 fev em 0:00 - 2 jun em 23:59 4 meses
Limite de tempo Nenhum **Tentativas permitidas** 2

Instruções

Importante:

Caso você esteja realizando a atividade através do aplicativo "Canvas Student", é necessário que você clique em "**FAZER O QUESTIONÁRIO**", no final da página.

Histórico de tentativas

	Tentativa	Tempo	Pontuação
MANTIDO	Tentativa 2	10 minutos	0,8 de 1
MAIS RECENTE	Tentativa 2	10 minutos	0,8 de 1
	Tentativa 1	40 minutos	0 de 1

Pontuação desta tentativa: **0,8** de 1

Enviado 4 mar em 20:20

Esta tentativa levou 10 minutos.

Pergunta 1

0,2 / 0,2 pts

Leia o trecho a seguir:

"Cada construtor de subclasse deve, implícita ou explicitamente, chamar um dos construtores da superclasse para inicializar as variáveis de instância herdadas da superclasse (...) Fazemos isso usando a sintaxe de chamada de construtor de superclasse — a palavra-chave `super` é seguida por um conjunto de parênteses que contém os argumentos do construtor da superclasse, que são usados para inicializar as variáveis de instância da superclasse. Se o construtor (...) não invocou o construtor da superclasse de maneira explícita, o compilador tentará inserir uma chamada para o construtor sem argumentos ou padrão da superclasse."

Fonte: DEITEL, H.; DEITEL, P. **Java**: como programar. 10. ed. São Paulo: Pearson Education do Brasil, 2017. p. 297. Adaptado.

Considerando as informações apresentadas, assinale a opção correta:



O construtor `super()` é uma sobreposição do construtor da classe base realizado na classe herdeira, ou seja, o construtor é reescrito para se adequar à realidade da classe herdeira.



Caso o construtor da classe base possua parâmetros, nós podemos chamar o construtor `super()` explicitamente. Caso contrário, sua chamada deverá ser feita implicitamente pelo compilador sem parâmetros.

Correto!

O construtor `super()` elimina a necessidade de copiar código do construtor da classe base dentro do construtor da classe herdeira, evitando repetição de código e melhorando a manutenção do código.

Alternativa correta.

O construtor `super()` representa uma chamada direta para o método construtor da classe base. Portanto, ele nos ajuda a economizar código: ao invés de copiar e colar suas funcionalidades, podemos realizar ações específicas no construtor da classe herdeira e chamar o construtor da classe base para realizar as ações genéricas comuns a todas as classes da “família”.

Construtores não possuem modificadores de acesso e não podem ser privados, portanto sempre são acessíveis.

Cada classe possui os seus próprios construtores, não existindo “sobreposição” por parte de uma classe herdeira. Os construtores da classe herdeira sempre irão utilizar também o construtor da classe base. Caso não haja uma chamada explícita, o compilador irá acrescentar essa chamada automaticamente – neste caso, ele utilizará o construtor padrão, que normalmente é o construtor sem parâmetros.



O construtor `super()` pode ser utilizado quando os atributos da classe base são públicos ou protegidos. Caso eles sejam privados, o construtor não terá acesso a eles, e por isso a chamada a `super()` provocará erros.



Para que o `super()` possa ser utilizado, é necessário que a classe base torne o seu construtor público ou protegido, caso contrário ele não será acessível dentro da classe herdeira.

Pergunta 2

0 / 0,2 pts

Analise os trechos de código abaixo:

```
class A {  
    /* ...  
    declarações de atributos e métodos  
    ... */  
}  
  
class B extends A {  
    /* ...  
    declarações de atributos e métodos  
    ... */  
}
```

O que podemos afirmar sobre a relação entre as classes A e B?

Resposta correta

☐

Objetos da classe B possuem todos os atributos e métodos da classe A, podendo possuir também outros métodos e atributos.

Você respondeu



Objetos da classe A possuem todos os atributos e métodos da classe B, podendo possuir também outros métodos e atributos.

Alternativa incorreta.

A classe B é herdeira de classe A. Com isso, todos os atributos e métodos de A automaticamente passam a existir para B. O oposto não necessariamente ocorre: a classe B pode implementar novos atributos e métodos, e eles não existirão para A. Além disso, a classe B pode sobrepor os métodos da classe B, ou seja, reescrever a funcionalidade de um método mantendo o mesmo nome. Por fim, o acesso de uma classe aos atributos da outra depende dos modificadores de acesso utilizados: a classe A só terá acesso a atributos públicos da classe B. A classe B, por ser herdeira de A, terá acesso a seus atributos públicos e também aos protegidos, mas não aos privados (mesmo possuindo esses atributos por herança).

- ☐ Métodos da classe B com o mesmo nome de métodos da classe A serão idênticos aos da classe A.
- ☐ Métodos da classe A possuem acesso a todos os atributos da classe B.
- ☐ Métodos da classe B possuem acesso a todos os atributos da classe A.

Pergunta 3

0,2 / 0,2 pts

Examine o código abaixo:

```
void funcao1 (Veiculo v) {  
    v.locomocao() ;  
}  
  
void funcao2 () {  
    Motocicleta m = new Motocicleta() ;  
    funcao1 (m) ;  
}
```

Sabendo que a classe Motocicleta é herdeira da classe base Veiculo, analise as asserções abaixo e a relação proposta entre elas:

I. Em funcao1 ocorre polimorfismo estático, ou seja, em tempo de compilação, quando a implementação do método a ser utilizada é escolhida durante a geração do bytecode.

PORQUE

II. Classes herdeiras podem sobrepor métodos da classe base, portanto a implementação do método utilizado em funcao1 irá depender da classe do objeto passado em “v”.

A respeito dessas asserções, assinale a opção correta:

-
- ☐ As asserções I e II são proposições verdadeiras, mas a asserção II não é uma justificativa correta da I.

Correto!

- ☐ As asserções I e II são proposições verdadeiras, e a asserção II é uma justificativa correta da I.
- ☒ A asserção I é uma proposição falsa e a asserção II é uma proposição verdadeira.

Alternativa correta.

O polimorfismo estático de fato ocorre em tempo de compilação (ou seja, a implementação do método é escolhida quando o bytecode é gerado). Porém, essa forma de polimorfismo ocorre quando um método é sobrecarregado, ou seja, quando em uma mesma classe temos várias implementações diferentes do mesmo método com diferentes parâmetros. O compilador decide a implementação a ser utilizada com base nos parâmetros passados.

O polimorfismo dinâmico ocorre em tempo de execução. Ele ocorre quando um método é chamado a partir de uma referência de objeto que pode ser de uma classe ou de suas classes herdeiras. Como classes herdeiras podem sobrepor métodos da classe base, durante a execução a classe do objeto passado para o ponteiro deve ser verificada para decidir se será utilizado o método original ou uma das sobreposições.

No exemplo dado, verificamos um ponteiro para um objeto em uma função, e outra função passa um objeto de uma classe herdeira. Portanto, a proposição I é falsa, pois não estamos lidando com polimorfismo estático, mas a proposição II é verdadeira, pois classes herdeiras podem sobrepor métodos e durante a execução, graças ao polimorfismo dinâmico, será decidida a implementação do método a ser utilizado com base na classe do objeto.

- ☐ A asserção I é uma proposição verdadeira e a asserção II é uma proposição falsa.
- ☐ Ambas as asserções são proposições falsas.

Pergunta 4**0,2 / 0,2 pts**

Leia o trecho a seguir:

"Para sobrescrever um método de superclasse, uma subclasse deve declarar um método com a mesma assinatura (nome de método, número de parâmetros, tipos de parâmetro e ordem dos tipos de parâmetro), como o método de superclasse."

Fonte: DEITEL, H.; DEITEL, P. **Java**: como programar. 10. ed. São Paulo: Pearson Education do Brasil, 2017. p. 289.

Analise as seguintes afirmações sobre a sobreposição de métodos:

- I. Um método não sobreposto preservará seu funcionamento independentemente da classe.
- II. Todo método público herdado deverá ser sobreposto; os privados, não.
- III. Uma classe derivada só pode criar métodos com os mesmos nomes e parâmetros que a classe base.

Está correto o que se afirma em:

☐ II e III, apenas.

Correto!

☒ I, apenas.

Alternativa correta.

Quando uma classe herda de outra classe, ela automaticamente recebe todos os métodos da classe original, e pode utilizá-los da forma que vieram. A afirmação I está correta.

É possível redefinir um método herdado, basta reescrevê-lo utilizando a mesma assinatura, mas com conteúdo diferente. Mas isso é opcional, e quando um método não for sobrescrito, será utilizado o método herdado da forma que estava na classe mãe. Se o método for público, ele poderá ser usado diretamente. Caso contrário, apenas a partir do “super”. A afirmação II está incorreta.

Uma classe derivada pode criar quantos e quais métodos novos forem necessários, sem restrições. Só é necessário utilizar as assinaturas dos métodos da classe mãe quando estamos especificamente realizando a sobrescrita (ou sobreposição) do método herdado. A afirmação III está incorreta.

☐ I e III, apenas.

☐ I e II, apenas.

☐ III, apenas.

Pergunta 5

0,2 / 0,2 pts

Leia o trecho a seguir:

"O operador *instanceof* determina se um objeto tem o relacionamento *é um* com um tipo específico (...) O relacionamento *é um* se aplica apenas entre a subclasse e suas superclasses, não vice-versa."

Fonte: DEITEL, H.; DEITEL, P. **Java**: como programar. 10. ed. São Paulo: Pearson Education do Brasil, 2017. p. 343. Adaptado.

Considerando as informações apresentadas e o operador *instanceof*, avalie as afirmativas a seguir:

- I. Seu resultado é verdadeiro quando testamos um objeto de uma certa classe com uma classe derivada de sua classe.
- II. Seu resultado é verdadeiro quando testamos um objeto de uma certa classe com uma classe da qual a sua deriva.
- III. Seu resultado é verdadeiro quando testamos um objeto de uma certa classe com a sua própria classe.
- IV. O uso de *instanceof* pode constituir uma violação do polimorfismo, por diferenciar objetos de diferentes classes.

Estão corretas as alternativas:

☐ I, III e IV, apenas.

☐ I, II e III, apenas.

☐ I, II e IV, apenas.

Correto!

☒ II, III e IV, apenas.

Alternativa correta.

Como o texto afirma, o relacionamento “é um” se aplica entre uma subclasse e suas superclasses, e não vice-versa. Portanto, se classe B é herdeira de classe A, um objeto da classe B também é instância da classe A, mas um objeto da classe A não é instância da classe B. A afirmativa I propõe que um objeto da classe A seria instância da classe B, o que está incorreto.

A afirmativa II propõe que o objeto da classe B também é instância da classe A, e por isso está correta.

A afirmativa III propõe que um objeto da classe B é instância da classe B, o que também está correto.

A afirmativa IV está correta, pois pelo princípio do polimorfismo, não importa a qual classe específica um objeto pertence. Se ele pertence a qualquer subclasse de uma certa classe, ele pode se comportar de maneira análoga a objetos daquela classe, pois possui todos os métodos disponíveis na classe base. Ao utilizarmos o *instanceof*, estamos tentando diferenciar objetos derivados de uma mesma classe, portanto violando o polimorfismo.

☐ I, II, III e IV.

Pontuação do teste: **0,8** de 1